ClassQuest -- Code Review

Capstone Project | April 2026 | Team ClassQuest

1. Overview

This is a code review written by the ClassQuest team as part of the capstone project. The purpose is to take an honest look at the codebase we built -- what we are proud of, what we know could be better, and why certain decisions were made the way they were. It is not meant to be a harsh critique; it is just a transparent look at the state of the project going into the live demo.

ClassQuest is a gamified classroom management platform. Teachers can create classes, assign quests, and run live boss battles where the whole class participates together. Students earn XP, level up their characters, and compete on a leaderboard. On the technical side, the frontend is built with React 19 and Vite, the backend runs on AWS Lambda through SST, data is stored in DynamoDB, and user accounts are managed through AWS Cognito.

2. How the Project is Structured

The project is split into two main parts that talk to each other through an API -- the frontend never directly touches the database, and the backend never cares about what the UI looks like. This is a clean separation and it made it a lot easier to work on both sides independently without breaking each other.

On the backend, every feature -- avatars, quests, boss battles, guilds, inventory, leaderboards -- follows the same structure. Each one has three files that do very specific jobs:

- handler.ts -- this is the entry point. It reads the request, checks the inputs, and sends back the response.
- repo.ts -- this is the only place that talks to the database. Nothing else does.
- validation.ts -- this checks whether the data coming in is valid before anything gets saved.

We stuck to this structure across all 80+ API endpoints. It means that if something breaks, you always know where to look -- either the handler read the request wrong, the validation let something through it should not have, or the database call failed. It also made writing tests a lot more straightforward.

For the boss battles specifically, we used AppSync instead of the regular REST API. This is because boss battles need to update everyone in the room at the same time -- when the teacher

advances a question or a student submits an answer, every screen needs to reflect that instantly. AppSync handles this through subscriptions, which is exactly what it was built for.

3. What Went Well

Looking back at the codebase, there are a few things we think came out genuinely well.

- We never let bad input reach the database. Every endpoint validates the request first and returns a helpful error message if something is wrong. This prevented a lot of silent bugs that would have been hard to track down later.
- The database design for the leaderboard is something we are particularly happy with. Instead of fetching all students and sorting them in code, we store an inverted XP value in DynamoDB and use it as a sort key. This means the leaderboard comes back already sorted, which is faster and simpler.
- The boss battle flow has a proper state machine built in. A battle can only move forward in the right order -- you cannot finish a battle that has not started, or advance a question that is already resolved. These checks are enforced at the backend level, not just in the UI.
- All API calls on the frontend go through one shared function in http.ts. This means the login token gets attached automatically to every request -- we never have to think about it in individual components. It also means error handling works the same way everywhere.
- The character equipment system has a really clean type setup in CharacterPage.tsx. There are mapping tables that connect the slot names in the UI to the field names in the database, and TypeScript will throw an error at compile time if a slot is missing from any of the maps. This saved us from a whole category of bugs.
- The backend tests are thorough. The test setup mocks the database at the SDK level, which means we are genuinely testing the business logic and not just checking that DynamoDB gets called.

4. Issues and Areas for Improvement

These are the things we noticed while reviewing the code. Some are security-related, some are code quality issues, and some are just things that were done a certain way during development that we would revisit if we had more time.

[Critical] The login token is read but never actually verified

When a user sends a request to the backend, they include a login token (called a JWT) that proves who they are. Our auth.ts file reads the information inside that token -- things like the user ID and whether they are a teacher or student -- but it never checks whether the token itself is genuine. In theory, someone could create a fake token with whatever information they want and our backend would trust it. There is actually a comment in the code that acknowledges this was left this way on purpose during development.

Fix: The fix is to turn on the JWT Authorizer in the API Gateway settings and point it at our Cognito user pool. This is a configuration change, not a code change -- AWS handles the verification automatically and rejects fake tokens before they even reach our Lambda functions.

[High] Teachers whose accounts are still pending approval have the same access as approved teachers

When a teacher signs up, their Cognito account gets added to a group called TeachersPending until they are approved. The problem is that our auth.ts file treats TeachersPending and Teachers as the same role. So a teacher who has not been approved yet can still create classes, launch boss battles, and manage students -- exactly the same as someone who has been fully approved.

Fix: The cleanest fix would be to build out the approval workflow so that pending teachers are actually restricted until they are approved. A simpler short-term fix would be to only give TeachersPending accounts read access -- they can log in and look around, but they cannot create or modify anything until approved.

[Medium] The way we store and read the current user is inconsistent across the frontend

When a user logs in, we save their information (name, role, class details) to something called localStorage, which is basically the browser's built-in storage. We store it under a key called cq_currentUser. The problem is that we then read this back in at least six different components, and each one has its own copy of the same code to do it. If we ever change the structure of that stored object, we have to update it in six places. On top of that, there is a hardcoded fallback in teacherDashboard.tsx that uses 'teacher-123' as a default teacher ID if the real one is not available -- which would create fake records in the database linked to an ID that does not exist.

Fix: The fix is to write one useCurrentUser() hook that handles reading from localStorage, and then import that hook wherever it is needed. Everything goes through one place. The 'teacher-123' fallback should just be removed -- if we do not have a real teacher ID, we should stop and show an error rather than proceed with a fake one.

[Medium] The URL paths in the app are not named consistently

Some of our page URLs use PascalCase, which is the style used for naming components in React -- for example /TeacherLogin and /StudentLogin. Others use kebab-case, which is the standard style for URLs on the web -- like /teacher/dashboard and /boss-lobby/:id. Mixing these two styles in the same app looks unfinished and makes it harder to remember which format a given route uses. We also have two separate redirect routes (/TeacherDashboard and /teacherDashboard) that both just point to /teacher/dashboard, which suggests these were added at different times without cleaning up the old ones.

Fix: Pick one style -- kebab-case is the right choice for URLs -- and update all the routes to match. Once all the internal links are pointing to the correct URLs, the duplicate redirects can be deleted.

[Low] The NPC characters in boss battles use hardcoded sprite images

During a boss battle, any empty spots in the character grid are filled with NPC characters. The images for these NPCs are hardcoded in a list of ten file paths directly in the bossFight file. This works fine right now, but if the asset folder ever gets reorganised or the image files get renamed, these references will silently break and the NPCs will just not appear.

Fix: The avatar base images are already available through the avatarBases API. We could just pull a random selection from there instead of hardcoding paths. That way the NPCs always use real, valid assets from the same source as player avatars.

[Low] Several Lambda functions are missing proper TypeScript types on the event parameter

A few of our backend functions -- mostly in the boss battle area -- are written with the incoming request typed as 'any'. In TypeScript, using 'any' basically turns off the type checker for that variable, which means the editor will not warn us if we try to access a property that does not exist. It is a small thing but it does make those files a bit riskier to edit without making mistakes.

Fix: Replace 'any' with the proper type, which is APIGatewayProxyEventV2. It is already available in the AWS SDK types package we are already using.

[Low] The auth helper file has no tests despite being used everywhere

The auth.ts file is imported by almost every protected endpoint in the backend. It is responsible for reading the login token and deciding what role the user has. Despite how

important it is, it has no unit tests at all -- so if someone makes a change to it, there is no automated way to check whether it still works correctly.

Fix: Adding tests to auth.ts would be a good starting point -- things like checking that expired tokens get rejected, that a missing token returns the right error, and that users in the right Cognito groups get the right role assigned.

5. Summary

Severity	Finding
Critical	Login tokens are read but never verified for authenticity
High	Pending teachers have the same access as approved teachers
Medium	Current user stored and read inconsistently across frontend
Medium	URL paths mix PascalCase and kebab-case styles
Low	NPC sprite images are hardcoded instead of fetched from the API
Low	Some Lambda functions use 'any' instead of proper TypeScript types
Low	No tests for the auth helper file

Overall we are happy with how ClassQuest turned out. The backend is consistent and well-structured, the database design is solid, and the boss battle feature works the way we envisioned it. The issues listed above are real and worth acknowledging -- the token verification one in particular is something we would address first in any future version. But none of them affect what the app does during the demo, and they are all fixable with focused effort. It was a good project.